

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – Comparative Analysis

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO4: Articulation (BL4, BL5)	Assessment:	Assessment Tool 3 – Comparative Analysis
Weightage:	10% of CIA	Total Marks:	100
CO4 Statement:	Solve optimization problems using dynamic programming and greedy strategies.		
Student Name:	Mohan Kumar J	Reg. No.:	192424060
Section / Batch:	2024 [AI&DS]	Date:	23-08-2026

Instructions to Students

- Answer the following question. Total Marks: 100.
- Analyze the given questions thoroughly before answering.
- Compare multiple aspects such as methods, results, advantages, and limitations clearly.
- Critically evaluate the strengths, weaknesses, and implications with proper justification.
- Present meaningful insights, interpretations, and well-supported conclusions from the comparisons.

Question Type	Focus Area	Questions
Comparative Analysis	Comparative analysis of Dynamic Programming and Greedy algorithms for optimization problems.	Q1

SECTION A – Comparative Analysis

Code of Conduct

I Mohan Kumar J (192424060) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student:

Mohan Kumar

RUBRICS FOR CALCULATION

Comparative Analysis					
Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Selection of Studies/Parameters	20%	Selects highly relevant and diverse studies with well-defined comparison parameters	Selects relevant studies with minor gaps in parameter definition	Limited or partially relevant selection	Poor or inappropriate selection of studies
Comparison & Differentiation	25%	Clearly compares multiple aspects (methods, results, limitations) with strong differentiation	Good comparison with minor gaps	Basic comparison with limited depth	No clear comparison; mostly descriptive
Critical Evaluation	25%	Critically evaluates strengths, weaknesses, and implications with strong justification	Provides evaluation with some justification	Limited evaluation; lacks depth	No critical evaluation provided
Synthesis & Insight Generation	20%	Generates meaningful insights and conclusions from comparisons	Some insights derived from comparison	Limited or unclear insights	No insights generated
Clarity	10%	Information is clearly presented (tables/figures) with logical structure	Generally clear with minor issues	Presentation lacks clarity or structure	Poor presentation and difficult to understand

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Selection of Studies/Parameters	20%				
Comparison & Differentiation	25%				
Critical Evaluation	25%				
Synthesis & Insight Generation	20%				
Clarity	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

TRAVELLING SALESPERSON PROBLEM COMPARISON

Dynamic Programming vs Greedy Heuristic

1. Introduction

The **Travelling Salesperson Problem (TSP)** is one of the most well-known problems in computer science, operations research, combinatorial optimization, and algorithm design. The objective of TSP is to determine the shortest possible route through a collection of cities such that every city is visited exactly once and the route finally returns to the starting city.

Although the problem statement is simple, TSP is computationally challenging because the number of possible routes increases extremely rapidly as the number of cities increases. For (n) cities, fixing the starting city still leaves approximately $((n-1)!)$ possible tours to consider using a brute-force approach.

TSP has several real-world applications, including:

- Delivery route optimization
- Logistics and transportation
- Warehouse management
- Robot navigation
- Circuit-board drilling
- Network routing
- Supply-chain optimization
- Vehicle routing
- Manufacturing operations
- Travel planning

In this analysis, two different algorithmic approaches are used to solve the TSP:

1. **Dynamic Programming (DP)** using the Held-Karp approach
2. **Nearest Neighbor Greedy Heuristic**

The main objective is to compare these algorithms in terms of:

- Total tour cost
- Solution quality
- Optimality
- Computational effort
- Time complexity

- Space complexity
- Scalability
- Strengths and weaknesses
- Practical applications

The analysis also explains why a greedy algorithm can produce a good-looking solution without guaranteeing the globally optimal solution and why the exponential complexity of Dynamic Programming creates scalability challenges.

2. Problem Statement

Given a set of four cities and their pairwise travelling distances, determine the minimum-cost tour that:

1. Starts from a selected city.
2. Visits every other city exactly once.
3. Returns to the starting city.

The problem must be solved using:

- **Dynamic Programming**
- **Nearest Neighbor Greedy Approach**

The results must then be compared based on total tour cost and computational effort.

The analysis should also explain:

- Why the greedy algorithm does not guarantee optimality.
- Why Dynamic Programming has exponential complexity.
- How the algorithms behave as the number of cities increases.
- Which approach is more appropriate for different practical situations.

3. Objectives

The major objectives of this study are:

Objective 1: Solve the TSP using Dynamic Programming

The DP approach should systematically evaluate relevant combinations of cities and determine the globally optimal tour.

Objective 2: Solve the TSP using a Greedy Heuristic

The Nearest Neighbor algorithm should construct a route by repeatedly selecting the nearest unvisited city.

Objective 3: Compare solution quality

The total cost obtained from both methods should be compared against each other.

Objective 4: Analyze computational complexity

The time and space requirements of both algorithms should be studied.

Objective 5: Critically evaluate the algorithms

The strengths, weaknesses, limitations, and practical suitability of both approaches should be discussed.

Objective 6: Analyze scalability

The effect of increasing the number of cities on computational requirements should be explained.

Objective 7: Generate meaningful insights

The final analysis should identify when an exact algorithm should be preferred and when a heuristic approach is more practical.

4. Selected Parameters for Comparison

To provide a meaningful comparison, the following parameters are considered.

Parameter	Dynamic Programming	Nearest Neighbor Greedy
Algorithm type	Exact optimization	Heuristic
Solution guarantee	Globally optimal	No guarantee
Main strategy	Dynamic programming/state-space optimization	Local nearest choice
Time complexity	$(O(n^2 2^n))$	$(O(n^2))$
Space complexity	$(O(n^2 n))$	$(O(n))$
Solution quality	Optimal	May be suboptimal
Computational effort	High	Low
Scalability	Limited	Good
Starting-point sensitivity	Low	High
Implementation difficulty	Moderate	Low
Best use case	Small/medium exact problems	Large approximate problems
Benchmark capability	Excellent	Limited

These parameters provide a diverse basis for evaluating both methods and satisfy the requirement of comparing multiple aspects rather than considering only the final distance.

5. Input Distance Matrix

The following four cities are considered:

- **A**
- **B**
- **C**
- **D**

The distance matrix is:

City	A	B	C	D
A	0	34	11	13
B	34	0	20	11
C	11	20	0	18
D	13	11	18	0

The matrix is symmetric:

$$d(A, B) = d(B, A) = 34$$

$$d(A, C) = d(C, A) = 11$$

$$d(A, D) = d(D, A) = 13$$

$$d(B, C) = d(C, B) = 20$$

$$d(B, D) = d(D, B) = 11$$

$$d(C, D) = d(D, C) = 18$$

The diagonal values are zero because the distance from a city to itself is zero.

The salesperson starts at **A** and must eventually return to **A**.

6. Travelling Salesperson Problem Representation

A valid TSP route must satisfy the following conditions:

- Every city must be visited.
- No city can be visited more than once before returning.

- The starting city must be revisited at the end.
- The total distance should be minimized.

For example:

[A B C D A]

is a valid tour.

Its cost is:

$$34+20+18+13 = 85$$

However, this is not necessarily the optimal tour.

The goal is to identify the route with the minimum possible cost.

7. Dynamic Programming Approach

7.1 Overview

Dynamic Programming is an **exact optimization technique**. Instead of simply making the cheapest immediate choice, it systematically evaluates different combinations of visited cities.

For TSP, the commonly used Dynamic Programming solution is based on the **Held-Karp algorithm**.

The major idea is to divide the original problem into smaller overlapping subproblems.

Suppose:

$$DP[S][j]$$

represents the minimum cost of:

- Starting at A
- Visiting all cities in subset (S)
- Ending at city (j)

The recurrence is:

$$DP[S][j] = \min_{i \in S, i \neq j} (DP[S - \{j\}][i] + d(i, j))$$

This means that before reaching city (j), the salesperson must have been at some other city (i).

The algorithm stores the best result for each state so that the same subproblem does not need to be solved repeatedly.

8. Dynamic Programming Initialization

The starting city is A.

For the first-level states:

$A \rightarrow B$

$$DP[\{B\}][B] = 34$$

$A \rightarrow C$

$$DP[\{C\}][C] = 11$$

$A \rightarrow D$

$$DP[\{D\}][D] = 13$$

Therefore:

Visited Set	Ending City	Cost
{B}	B	34
{C}	C	11
{D}	D	13

9. Dynamic Programming: Two-City States

9.1 Visiting B and C

There are two possible ways to reach B after visiting C:

[ACB]

Cost:

$$11 + 20 = 31$$

Therefore:

$$DP[\{B, C\}][B] = 31$$

To finish at C:

[ABC]

Cost:

$$34 + 20 = 54$$

Therefore:

$$DP[\{B, C\}][C] = 54$$

9.2 Visiting B and D

To finish at B:

[ADB]

Cost:

$$13 + 11 = 24$$

Therefore:

$$DP[\{B, D\}][B] = 24$$

To finish at D:

[ABD]

Cost:

$$34 + 11 = 45$$

Therefore:

$$DP[\{B, D\}][D] = 45$$

9.3 Visiting C and D

To finish at C:

[ADC]

Cost:

$$13 + 18 = 31$$

Therefore:

$$DP[\{C, D\}][C] = 31$$

To finish at D:

[ACD]

Cost:

$$11 + 18 = 29$$

Therefore:

$$DP[\{C, D\}][D] = 29$$

The complete two-city table is:

Visited Cities	Ending City	Minimum Cost
{B,C}	B	31
{B,C}	C	54
{B,D}	B	24
{B,D}	D	45
{C,D}	C	31
{C,D}	D	29

10. Dynamic Programming: Complete State

Now all three non-starting cities must be visited:

$$\{B, C, D\}$$

There are three possible ending cities.

10.1 Ending at B

The possible previous cities are C and D.

Through C:

[ACDB]

Cost:

$$11 + 18 + 11 = 40$$

Through D:

[ADCB]

Cost:

$$13 + 18 + 20 = 51$$

Therefore:

$$DP[\{B, C, D\}][B] = 40$$

10.2 Ending at C

Possible previous cities are B and D.

Through B:

[ADBC]

Cost:

$$13+11+20=44$$

Through D:

[ABDC]

Cost:

$$34+11+18=63$$

Therefore:

$$DP[\{B, C, D\}] [C] = 44$$

10.3 Ending at D

Possible previous cities are B and C.

Through B:

[ACBD]

Cost:

$$11+20+11=42$$

Through C:

[ABCD]

Cost:

$$34+20+18=72$$

Therefore:

$$DP[\{B, C, D\}] [D] = 42$$

The final DP state is:

All Cities Visited	Ending City	Minimum Cost
{B,C,D}	B	40
{B,C,D}	C	44
{B,C,D}	D	42

11. Nearest Neighbor Greedy Approach

11.1 Concept

The Nearest Neighbor algorithm is a **greedy heuristic**.

It does not attempt to calculate every possible route. Instead, it follows a simple rule:

At every step, visit the nearest unvisited city.

The algorithm is:

1. Start at A.
2. Find the nearest unvisited city.
3. Travel to that city.
4. Mark it as visited.
5. Repeat until every city has been visited.
6. Return to A.

The algorithm is computationally simple and fast.

However, its major limitation is that it makes **local decisions** without considering the complete future route.

12. Greedy Step-by-Step Calculation

Step 1: Start at A

Distances from A are:

Destination	Distance
B	34
C	11
D	13

The nearest city is C.

Therefore:

[AC]

Cost: 11

Cumulative cost: 11

Step 2: From C

Unvisited cities:

- B
- D

Distances:

Destination	Distance
B	20
D	18

The nearest city is D.

Therefore:

[CD]

Cost: 18

Cumulative cost: $11+18=29$

Step 3: From D

Only B remains.

Therefore:

[DB]

Cost: 11

Cumulative cost: $29+11=40$

Step 4: Return to A

The salesperson must return to the starting city.

[BA]

Cost: 34

Therefore:

Total=40+34= 74

The greedy route is:

$A \rightarrow C \rightarrow D \rightarrow B \rightarrow A$

with total cost: 74

13. DP vs Greedy Result

The two algorithms produce:

Algorithm	Tour	Total Cost
Dynamic Programming	$A \rightarrow C \rightarrow B \rightarrow D \rightarrow A$	55
Nearest Neighbor	$A \rightarrow C \rightarrow D \rightarrow B \rightarrow A$	74

The difference is:

$74-55=19$

Therefore, the greedy algorithm is 19 distance units more expensive.

The percentage difference relative to the optimal solution is:

$$\begin{aligned} &= \frac{74-55}{55} \times 100 \\ &= 0.34545 \times 100 \\ &= 34.55\% \end{aligned}$$

Thus, in this particular example, the greedy solution is approximately **34.55% more expensive** than the optimal solution.

14. Time Complexity Analysis

14.1 Dynamic Programming

The Held-Karp Dynamic Programming approach has the time complexity:

$$O(n^2 2^n)$$

The (2^n) term occurs because the algorithm considers subsets of cities.

Its space complexity is:

$$O(n 2^n)$$

because the DP table stores information for many subsets and ending cities.

The major problem is the exponential component:

$$2^n$$

As (n) increases, the number of states grows extremely quickly.

15. Greedy Complexity

For Nearest Neighbor, the algorithm repeatedly searches for the nearest unvisited city.

At each stage, it may examine up to (n) cities.

This process occurs approximately (n) times.

Therefore: $O(n^2)$

time complexity.

The memory requirement is approximately:

$$O(n)$$

because only the visited information and tour need to be maintained.

Therefore, the greedy approach is significantly more scalable than Dynamic Programming.

16. Complexity Growth

The difference between the algorithms becomes obvious as the number of cities increases.

Number of Cities	(2^n)	DP Complexity Trend	Greedy Complexity Trend
4	16	Very small	Very small
5	32	Small	Small
10	1,024	Moderate	Low
15	32,768	High	Low
20	1,048,576	Very high	Low
25	33,554,432	Extremely high	Moderate
30	1,073,741,824	Very high	Practical
40	1,099,511,627,776	Impractical	Practical

For example:

[$2^{\{30\}}=1,073,741,824$]

possible subsets exist for 30 cities.

This demonstrates why exact Dynamic Programming cannot simply be scaled to very large TSP instances.

17. Limitations of This Experiment

Although the comparison clearly demonstrates the difference between the algorithms, there are some limitations.

31.1 Small Dataset

Only four cities are used.

Therefore, actual runtime differences are not representative of large TSP instances.

31.2 Symmetric Distances

The matrix assumes:

[$d(i,j)=d(j,i)$]

Real transportation networks may have asymmetric distances due to:

- One-way roads
- Traffic
- Different travel times
- Road restrictions

31.3 Static Distances

The experiment assumes that distances do not change.

Real-world routing may involve:

- Traffic congestion
- Weather
- Road closures
- Time-dependent travel costs

31.4 Basic Greedy Strategy

Only the basic Nearest Neighbor heuristic is evaluated.

More sophisticated heuristics could potentially produce much better routes.

These limitations should be considered before applying the experimental results directly to real-world transportation systems.

18. Overall Findings

The main findings are summarized below.

Finding	Result
Optimal DP tour	$A \rightarrow C \rightarrow B \rightarrow D \rightarrow A$
Optimal DP cost	55
Greedy tour	$A \rightarrow C \rightarrow D \rightarrow B \rightarrow A$
Greedy cost	74
Cost difference	19
Greedy gap	34.55%
DP time complexity	$(O(n^2 2^n))$
Greedy time complexity	$(O(n^2))$
DP optimality	Guaranteed
Greedy optimality	Not guaranteed
DP scalability	Limited
Greedy scalability	High

The results strongly support the conclusion that the two approaches represent different algorithmic priorities

19. Conclusion

The Travelling Salesperson Problem demonstrates the important trade-off between solution quality and computational efficiency.

For the given four-city distance matrix, the Dynamic Programming approach finds the optimal route:

$$\mathbf{A \rightarrow C \rightarrow B \rightarrow D \rightarrow A}$$

with a total cost of: 55

The Nearest Neighbor Greedy algorithm produces the route:

$$\mathbf{A \rightarrow C \rightarrow D \rightarrow B \rightarrow A}$$

with a total cost of: 74

Therefore, the greedy solution is 19 distance units more expensive, which is approximately 34.55% higher than the optimal solution.

Dynamic Programming guarantees the optimal solution, but its time complexity of $(O(n^2 2^n))$ and space complexity of $(O(n 2^n))$ make it computationally expensive for large numbers of cities. In contrast, the Nearest Neighbor algorithm has a time complexity of $(O(n^2))$, making it faster and more scalable, but it does not guarantee an optimal solution.

Therefore, Dynamic Programming is suitable for smaller problems where optimality is important, while the Nearest Neighbor Greedy approach is more suitable for larger problems where speed and computational efficiency are the main priorities.

SIMATS Online Programming Lab

DAA PROGRAMMING

MOHAN KUMAR
192424060RA

```

1 INF = float('inf')
2 n = 4
3 START = 0
4 dist = []
5 for i in range(n):
6     row = list(map(int, input().split()))
7     if len(row) != n:
8         raise ValueError("Each row must contain exactly 4 integers.")
9     dist.append(row)
10 dp = [[INF] * n for _ in range(1 << n)]
11 dp[1 << START][START] = 0
12 dp_ops = 0
13 for mask in range(1 << n):
14     for u in range(n):
15         if not (mask & (1 << u)):
16             continue
17         if dp[mask][u] == INF:
18             continue
19         for v in range(n):
20             if mask & (1 << v):
21                 continue
22             new_mask = mask | (1 << v)
23             new_cost = dp[mask][u] + dist[u][v]
24             if new_cost < dp[new_mask][v]:
25                 dp[new_mask][v] = new_cost
26             dp_ops += 1
27 full_mask = (1 << n) - 1
28 best_cost = INF
29 best_end = -1
30 for i in range(n):
31     if i == START:
32         continue
33     cost = dp[full_mask][i] + dist[i][START]
34     if cost < best_cost:
35         best_cost = cost
36         best_end = i
37 path = [best_end]
38 mask = full_mask
39 cur = best_end
40 while cur != START:
41     for prev in range(n):
42         if prev == cur:
43             continue
44         if mask & (1 << prev):
45             prev_mask = mask ^ (1 << cur)
46             if dp[prev_mask][prev] + dist[prev][cur] == dp[mask][cur]:
47                 path.append(prev)
48                 cur = prev
49                 mask = prev_mask
50                 break
51 path.append(START)
52 path.reverse()
53 dp_tour = path
54 def nearest_neighbor(start):
55     visited = [False] * n
56     tour = [start]
57     visited[start] = True
58     current = start
59     total_cost = 0
60     ops = 0
61     for _ in range(n - 1):
62         best_city = None
63         best_dist = INF
64         for v in range(n):
65             if not visited[v] and dist[current][v] < best_dist:
66                 best_dist = dist[current][v]
67                 best_city = v
68             ops += 1

```

```

0 10 15 20
10 0 35 25
15 35 0 30
20 25 30 0

```

Output

```

===== Nearest Neighbor Greedy =====
Tour : [0, 1, 3, 2, 0]
Cost : 80
Operations (approx): 12

```